

# La posta, scritta a mano

Un server Postfix e Dovecot, due macchine client, e un messaggio spinto attraverso entrambi i protocolli una riga per volta — SMTP per consegnarlo, IMAP per rileggerlo, e in mezzo il log del server. Da un lab in esecuzione.

---

**Comando** `qlab run mail-lab`

**VM** `mail-lab-server` · `mail-lab-client1` (alice) · `mail-lab-client2` (bob)

**LAN** `192.168.100.0/24`, isolata fra le tre VM

**Credenziali** `labuser` / `labpass` · utenti di posta `alice`, `bob` / `labpass`

**Risultato** `qlab test mail-lab` → 6 esercizi, 39 controlli, tutti superati

## 1. Tre macchine, due protocolli

```
mail-lab-server      192.168.100.1/24
mail-lab-client1     192.168.100.2/24
mail-lab-client2     192.168.100.3/24
```

```
mail-lab-server
alice:1001:/home/alice
bob:1002:/home/bob
```

Un server e due client, uno per utente. La separazione è voluta: con alice e bob su macchine diverse ogni messaggio deve attraversare un cavo, ed entrambe le metà del lavoro — consegnare un messaggio e andarselo a riprendere — si vedono come traffico di rete e non come file che compaiono in una directory.

alice e bob sono normali account Unix sul server, con normali home. Il database utenti è tutto lì. Lo leggono sia Postfix sia Dovecot, ed è il motivo per cui questo lab non ha bisogno di LDAP, di SQL né di tabelle di mailbox virtuali.

## 2. Cosa gira sul server

```
$ the two daemons, and the ports they hold
active active
0.0.0.0:993 "dovecot",pid=6303,fd=34
0.0.0.0:25 "master",pid=6200,fd=13
0.0.0.0:143 "dovecot",pid=6303,fd=33
[::]:993 "dovecot",pid=6303,fd=35

$ postconf -n      (only what differs from the defaults)
home_mailbox = Maildir/
inet_interfaces = all
inet_protocols = ipv4
mailbox_size_limit = 0
mydestination = $myhostname, mail.lab, localhost.localdomain, localhost
mydomain = mail.lab
myhostname = mail.lab
mynetworks = 127.0.0.0/8, 192.168.100.0/24
myorigin = $mydomain
smtpd_banner = $myhostname ESMTP mail.lab
smtpd_relay_restrictions = permit_mynetworks, reject_unauth_destination

$ where Dovecot expects to find the mail
mail_location = maildir:~/Maildir
protocols = imap
auth_mechanisms = plain login
disable_plaintext_auth = no
```

Due demoni, quattro porte, e una divisione del lavoro netta: **Postfix possiede la porta 25** e sposta la posta fra macchine; **Dovecot possiede la 143 e la 993** e consegna la posta alle persone. Fra loro non si parlano mai. L'unico oggetto condiviso è la Maildir nella home di ogni utente: uno la scrive, l'altro la legge.

`postconf -n` stampa solo le impostazioni diverse dai default compilati, ed è il modo più veloce per vedere cosa è stato davvero detto a un server di posta. `myhostname` e `mydomain` gli danno un nome; `mydestination` elenca i domini che considera propri; `home_mailbox = Maildir/` sceglie il formato di memorizzazione; `mynetworks` decide chi può spedire attraverso di lui — la sezione 7 ci torna sopra.

Sul lato Dovecot, `mail_location` punta alla stessa Maildir, e `disable_plaintext_auth = no` è una comodità da laboratorio: permette di fare login sulla 143 in chiaro con `telnet` e guardare, che è esattamente quello che fa la sezione 6.

### 3. SMTP, una riga per volta

```
$ telnet 192.168.100.1 25      - the whole submission, typed by hand
Trying 192.168.100.1...
Connected to 192.168.100.1.
Escape character is '^J'.
220 mail.lab ESMTP mail.lab
250-mail.lab
250-PIPELINING
250-SIZE 10240000
250-VRFY
250-ETRN
250-ENHANCEDSTATUSCODES
250-8BITMIME
250-DSN
250-SMTPUTF8
250 CHUNKING
250 2.1.0 Ok
250 2.1.5 Ok
354 End data with <CR><LF>.<CR><LF>
250 2.0.0 Ok: queued as E517414B0
221 2.0.0 Bye
Connection closed by foreign host.
```

Questa è una consegna di posta completa senza niente fra le dita e il socket. Ogni riga che manda il client è un comando breve; ogni riga che risponde il server comincia con un codice di tre cifre.

La forma è sempre la stessa. `EHLO` annuncia il client e chiede cosa supporta il server — le righe `250-` sono quella risposta, un elenco di capacità che termina con un `250` (spazio, non trattino) sull'ultima. `MAIL FROM` apre una busta. `RCPT TO` ci mette sopra un destinatario, uno per riga. `DATA` passa dai comandi al contenuto, e il contenuto finisce con un punto solo su una riga: per questo il `354` sillaba il terminatore. Poi il server si assume la responsabilità:

**250 Ok: queued as ...** vuol dire che il messaggio è su disco e il mittente può dimenticarsene.

La busta e gli header sono cose diverse. `MAIL FROM` / `RCPT TO` sono la busta, ed è lei che instrada davvero il messaggio; `From`, `To` e `Subject` si scrivono dentro `DATA` e sono solo testo. Nessuno controlla che siano d'accordo — in quella distanza vivono i mittenti falsificati, ed è il motivo per cui SPF, DKIM e DMARC sono dovuti nascere sopra.

## 4. Lo stesso messaggio, visto dal server

```
$ the life of that one message, on the server
queue id: E517414B0

Sep 14 14:02:54 postfix/smtpd[6927]: E517414B0: client=unknown[192.168.100.2]
Sep 14 14:02:56 postfix/cleanup[6930]: E517414B0: message-id=<>
Sep 14 14:02:56 postfix/qmgr[6203]: E517414B0: from=<alice@mail.lab>, size=260, nrcpt=1 (queue active)
Sep 14 14:02:56 postfix/local[6931]: E517414B0: to=<bob@mail.lab>, relay=local, delay=3, delays=3/0/0/0,
dsn=2.0.0, status=sent (delivered to maildir)
Sep 14 14:02:56 postfix/qmgr[6203]: E517414B0: removed
```

Quel queue ID è il filo da tirare. Ogni riga che Postfix scrive su un messaggio se lo porta dietro, quindi cercare un solo ID restituisce tutta la vita del messaggio in ordine.

Cinque righe, cinque processi, e ognuno è un programma a sé: `smtpd` ha preso la connessione, `cleanup` ha normalizzato gli header e assegnato un message-id, `qmgr` l'ha messo in coda attiva, `local` l'ha consegnato e `qmgr` l'ha tolto. Postfix è costruito così apposta — programmi piccoli, una coda, privilegi minimi — e il log è il riflesso diretto di quell'architettura.

La riga da imparare a memoria è quella della consegna. `to=`, `relay=`, `delay=`, `dsn=`, `status=`: quando la posta sparisce, è questa riga che dice se il tuo server ce l'ha mai avuta, cosa ne ha fatto e di chi è la colpa.

## 5. Maildir: un file per messaggio

```
$ where it actually landed
/home/bob/Maildir
/home/bob/Maildir/cur
/home/bob/Maildir/new
/home/bob/Maildir/tmp

new 1 file(s)
cur 2 file(s)

14:02:56.9645628880 337 1789394576.Vfe01I14b7M967374.mail-lab-server

$ the message as stored – the headers Postfix added are at the top
Return-Path: <alice@mail.lab>
X-Original-To: bob@mail.lab
Delivered-To: bob@mail.lab
Received: from client1.lab (unknown [192.168.100.2])
    by mail.lab (Postfix) with ESMTP id E517414B0
    for <bob@mail.lab>; Mon, 14 Sep 2026 14:02:53 +0000 (UTC)
Subject: sent by hand
From: alice@mail.lab
To: bob@mail.lab

Typed straight into the socket.
```

`status=sent (delivered to maildir)` significava questo. Tre directory, un file per messaggio, nessun lock e nessun indice condiviso: il formato Maildir è tutto qui, ed è il motivo per cui sopravvive ai crash e a NFS dove un singolo file `mbx` non sopravvive.

Lo stato lo portano le sottodirectory. Una consegna nuova viene scritta in `tmp` e poi spostata in `new`; quando un client apre la mailbox il messaggio passa in `cur` e i suoi flag vengono appesi al nome del file. I conteggi qui sopra non sono decorazione: in `new` c'è quello che nessuno ha mai guardato, in `cur` quello che è già stato guardato. I messaggi precedenti di bob sono già in `cur` perché la sessione IMAP della sezione 6 gli ha aperto la INBOX.

In cima al file memorizzato ci sono gli header che Postfix ha aggiunto al passaggio. Il più importante è `Received:`, che registra quale host ha consegnato, quale ha preso in carico e quando. Un messaggio vero ne accumula uno per salto, e la pila si legge dal basso verso l'alto.

## 6. Anche IMAP, una riga per volta

```
$ telnet 192.168.100.1 143      - bob reads it, also by hand
Trying 192.168.100.1...
Connected to 192.168.100.1.
Escape character is '^]'.
* OK Waiting for authentication process to respond..
* OK [CAPABILITY IMAP4rev1 SASL-IR LOGIN-REFERRALS ID ENABLE IDLE LITERAL+ STARTTLS AUTH=PLAIN
AUTH=LOGIN] Dovecot (Ubuntu) ready.
a OK [CAPABILITY IMAP4rev1 SASL-IR LOGIN-REFERRALS ID ENABLE IDLE SORT SORT=DISPLAY THREAD=REFERENCES
THREAD=REFS THREAD=ORDEREDSUBJECT MULTIAPPEND URL-PARTIAL CATENATE UNSELECT CHILDREN NAMESPACE UIDPLUS
LIST-EXTENDED I18NLEVEL=1 CONDSTORE QRESYNC ESEARCH ESORT SEARCHRES WITHIN CONTEXT=SEARCH LIST-STATUS
BINARY MOVE SNIPPET=FUZZY PREVIEW=FUZZY PREVIEW STATUS=SIZE SAVEDATE LITERAL+ NOTIFY SPECIAL-USE] Logged
in
* LIST (\HasNoChildren) "." INBOX
b OK List completed (0.002 + 0.000 + 0.001 secs).
* FLAGS (\Answered \Flagged \Deleted \Seen \Draft)
* OK [PERMANENTFLAGS (\Answered \Flagged \Deleted \Seen \Draft \*)] Flags permitted.
* 3 EXISTS
* 1 RECENT
* OK [UNSEEN 1] First unseen.
* OK [UIDVALIDITY 1789394123] UIDs valid
* OK [UIDNEXT 5] Predicted next UID
c OK [READ-WRITE] Select completed (0.016 + 0.000 + 0.015 secs).
* 1 FETCH (FLAGS () BODY[HEADER.FIELDS (FROM SUBJECT)] {47}
Subject: sent by hand
From: alice@mail.lab

)
* 2 FETCH (FLAGS () BODY[HEADER.FIELDS (FROM SUBJECT)] {47}
Subject: sent by hand
From: alice@mail.lab

)
* 3 FETCH (FLAGS (\Recent) BODY[HEADER.FIELDS (FROM SUBJECT)] {47}
Subject: sent by hand
From: alice@mail.lab

)
d OK Fetch completed (0.004 + 0.000 + 0.003 secs).
* BYE Logging out
e OK Logout completed (0.001 + 0.000 secs).
Connection closed by foreign host.
```

IMAP ha un aspetto diverso da SMTP perché risolve un problema di forma diversa: non "prenditi questo messaggio" ma "fammi lavorare su una mailbox che resta sul tuo disco". Ogni comando del client ha un tag (`a`, `b`, `c` ...) e la risposta finale del server a quel comando lo ripete, così

più comandi possono essere in volo insieme. Le righe che cominciano con `*` sono dati non taggati che il server offre strada facendo.

`LOGIN` autentica e riceve in risposta un elenco di capacità. `LIST` elenca le mailbox. `SELECT INBOX` ne apre una e subito racconta cosa c'è dentro: `EXISTS`, `RECENT`, i flag che la mailbox supporta e `UIDVALIDITY`, il numero che un client guarda per decidere se la sua copia in cache è ancora affidabile. `FETCH` poi chiede parti precise di messaggi precisi: qui solo due campi di header, che è esattamente ciò che serve a un client per disegnare l'indice senza scaricare niente.

`BODY.PEEK[...]` invece di `BODY[...]` non è un dettaglio. Il `BODY[]` semplice imposta il flag `\Seen` come effetto collaterale della lettura; `PEEK` no. Ogni client di posta che hai usato fa questa scelta al posto tuo.

## 7. Chi può spedire attraverso questo server

```
$ who is allowed to post through this server
mynetworks = 127.0.0.0/8, 192.168.100.0/24
smtpd_relay_restrictions = permit_mynetworks, reject_unauth_destination
smtpd_recipient_restrictions =

$ two recipients offered from a LAN client, one local and one not
220 mail.lab ESMTP mail.lab
250 CHUNKING
250 2.1.0 Ok
550 5.1.1 <ghost@mail.lab>: Recipient address rejected: User unknown in local recipient table
250 2.1.5 Ok
354 End data with <CR><LF>.<CR><LF>
250 2.0.0 Ok: queued as 91DF1AF2A
221 2.0.0 Bye

$ and what became of the one that was accepted for off-site delivery
Sep 14 14:05:03 postfix/smtpd[6927]: 91DF1AF2A: client=unknown[192.168.100.2]
Sep 14 14:05:05 postfix/cleanup[6930]: 91DF1AF2A: message-id=<>
Sep 14 14:05:05 postfix/qmgr[6203]: 91DF1AF2A: from=<alice@mail.lab>, size=213, nrcpt=1 (queue active)
Sep 14 14:05:05 postfix/smtp[7028]: 91DF1AF2A: to=<nobody@example.org>, relay=none, delay=4.1,
delays=4.1/0/0/0, dsn=5.1.0, status=bounced (Domain example.org does not accept mail (nullMX))
Sep 14 14:05:05 postfix/bounce[7029]: 91DF1AF2A: sender non-delivery notification: 9E2EAAFEE
Sep 14 14:05:05 postfix/qmgr[6203]: 91DF1AF2A: removed

messages in alice INBOX now: 1
hdr.from: MAILER-DAEMON@mail.lab (Mail Delivery System)
hdr.subject: Undelivered Mail Returned to Sender
```

Due destinatari proposti nella stessa sessione, due risposte diverse, e la differenza è l'impostazione più importante di un server di posta.

`ghost@mail.lab` viene rifiutato con **550 ... User unknown in local recipient table**: il dominio è fra i `mydestination`, quindi il server è autoritativo e sa che quell'utente non c'è. `nobody@example.org` invece viene *accettato* — perché il client sta dentro `mynetworks` e `smtpd_relay_restrictions` comincia con `permit_mynetworks`. Il server ha accettato di portare posta verso un dominio che non è suo. Questo è il relay, ed è una cosa normale e voluta per le macchine di cui ti fidi.

`reject_unauth_destination` è ciò che impedisce che diventi un **open relay**: rifiuta lo stesso destinatario a chiunque stia fuori da `mynetworks`. Sbagliare questa coppia è il modo classico con cui un server di posta nuovo finisce su tutte le blocklist nel giro di un giorno.

Poi il messaggio accettato non va da nessuna parte, e il log spiega perché: nessuna rotta verso la destinazione, `status=bounced`, e Postfix genera una notifica di mancata consegna verso il mittente. La INBOX di alice ora contiene un messaggio, da `MAILER-DAEMON`. Accettare un messaggio e consegnarlo sono due promesse diverse, e il bounce è il server che mantiene la seconda quando non è riuscito a mantenerla.

## 8. Il client fa esattamente questo, con un'interfaccia

The human client does exactly the two things done by hand above:

```
--- alice (client1) ~/.muttrc ---
set from = "alice@mail.lab"
set folder = "imap://alice:labpass@mail.lab:143/"
set spoolfile = "imap://alice:labpass@mail.lab:143/INBOX"
set smtp_url = "smtp://alice@mail.lab:25/"
--- bob (client2) ~/.muttrc ---
set from = "bob@mail.lab"
set folder = "imap://bob:labpass@mail.lab:143/"
set spoolfile = "imap://bob:labpass@mail.lab:143/INBOX"
set smtp_url = "smtp://bob@mail.lab:25/"
```

\$ and the same mailbox, asked of Dovecot from the server side  
INBOX

messages in bob INBOX: 3

Quattro righe di configurazione di `mutt`, e corrispondono una a una alle due sessioni qui sopra: `folder` e `spoolfile` sono il lato IMAP, `smtp_url` è il lato SMTP. Un client di posta è due client di protocollo in un trench — uno per spedire, uno per leggere — e non devono nemmeno puntare allo stesso server.

`doveadm` è la vista da dietro: la stessa mailbox, gli stessi messaggi, letti direttamente dal disco senza rete in mezzo.

## 9. Provaci

```
qlab run mail-lab
qlab shell mail-lab-client1      # alice
qlab shell mail-lab-client2      # bob
```

Il percorso previsto è quello umano — `sudo -u alice bash`, poi `mutt` — ma quello che insegna è il precedente:

```
telnet 192.168.100.1 25      # EHLO / MAIL FROM / RCPT TO / DATA / .
telnet 192.168.100.1 143    # a LOGIN bob labpass / b SELECT INBOX / c FETCH 1 ...
```

Tre cose che vale la pena fare:

- Lanciare `sudo tail -f /var/log/mail.log` sul server in una finestra mentre scrivi una sessione SMTP in un'altra. Ogni comando che digiti compare lì.
- Spedire un messaggio con un header `From:` diverso dal `MAIL FROM`, poi leggerlo in mutt e vedere quale dei due ti mostra.
- Guardare un file passare da `new/` a `cur/` nell'istante in cui apri la mailbox.

```
Exercise 1 – Mail Server Anatomy
  Exercise 1: All 12 checks passed
Exercise 2 – Send and Receive Mail
  Exercise 2: All 4 checks passed
Exercise 3 – SMTP Protocol
  Exercise 3: All 5 checks passed
Exercise 4 – IMAP Protocol
  Exercise 4: All 5 checks passed
Exercise 5 – Postfix Configuration
  Exercise 5: All 9 checks passed
Exercise 6 – Add a New User
  Exercise 6: All 4 checks passed
All 6 exercise(s) passed!
```

`qlab test mail-lab` esegue 39 controlli sulle tre VM — compresa una conversazione SMTP completa, un login IMAP e la creazione da zero di un terzo utente, per dimostrare che "aggiungere un utente di posta" vuol dire davvero `adduser`.